
CASE STUDY REPORT

TITLE

Food Delivery Application: API Testing and Error Handling for Web Services

Rajvirsinh Dodiya [92400120478]

Harsh Kanjariya [92400120494]

Arun Busa [92400120614]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026-2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS

5EV4

BATCH

A

CASE STUDY REPORT

RUBRICS FOR CASE STUDY

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis, Cloud Concepts & SDG Mapping	4 Marks: Clear problem identification, appropriate cloud concepts, and accurate SDG mapping.	3 Marks: Good analysis with minor gaps in concepts or SDG mapping.	1-2 Marks: Basic problem understanding with partial cloud concepts or SDG mapping.	0 Marks: Inadequate or incorrect analysis and no SDG mapping.	4
2	Security Analysis & Technical Depth	4 Marks: Comprehensive security analysis, well-defined architecture, appropriate technologies, and justified solutions.	3 Marks: Good technical explanation with minor omissions.	1-2 Marks: Limited security analysis and technical details.	0 Marks: No meaningful security analysis or technical explanation.	4
3	Report Quality & References	4 Marks: Well-structured report in IEEE format with comprehensive technical content, proper citations, and relevant references.	3 Marks: Good report structure with minor formatting or referencing issues.	1-2 Marks: Basic report with limited technical details or incomplete references.	0 Marks: Incomplete report with poor documentation and missing references.	4
Total						12

Sr. No.	Case Study Title / Problem Statement	COs	Date	R1(4)	R2(4)	R3(4)	Marks (12)	Signature
1	Food Delivery Application: API Testing and Error Handling for Web Services	CO5: Open-source testing frameworks						

Ms. Priyanka Mangi
Subject Faculty

CASE STUDY REPORT

Food Delivery Application : API Testing and Error Handling for Web Services

Harsh Kanjariya

*Department of Computer Science and
Engineering*

Marwadi University

Rajkot, Gujarat, India

harsh.kanjariya133257@marwadiuniversity.ac.in

Rajvirsinh Dodiya

*Department of Computer Science and
Engineering*

Marwadi University

Rajkot, Gujarat, India

rajvirsinhdodiya.130090@marwadiuniversity.ac.in

Arun Busa

*Department of Computer Science and
Engineering*

Marwadi University

Rajkot, Gujarat, India

arun.busa131854@marwadiuniversity.ac.in

Abstract— The contemporary on-demand food delivery ecosystem relies extensively on distributed web service architectures to facilitate real-time, asynchronous interactions among customers, restaurant operators, delivery couriers, and central platform administrators. In this distributed paradigm, Representational State Transfer (REST) Application Programming Interfaces (APIs) serve as the foundational communication bridge across heterogeneous client applications and backend data services. However, the operational reliability and resilience of food delivery platforms are frequently compromised by client-side payload discrepancies, malformed data types, missing required attributes, broken authentication states, unauthorized horizontal privilege escalations, and unhandled database exceptions. This case study proposes a comprehensive, cloud-neutral software engineering architecture and validation framework for a representative food delivery platform developed using Node.js, Express.js, and MongoDB. The technical focus centers on systematic REST API testing, rigorous multi-tier input validation, and centralized error handling to safeguard software reliability. Leveraging Postman as an enterprise API testing harness, we formulate structured testing methodologies encompassing positive validation, negative boundary probing, role-based authorization verification, and automated regression suites. Furthermore, the proposed architecture incorporates defensive mechanisms including token-based authentication (JSON Web Tokens), granular role guards, deterministic error serialization, and rate-limiting controls to prevent sensitive information disclosure and catastrophic service interruptions. Finally, the study maps these technical enhancements to the United Nations Sustainable

Development Goal 9 (SDG 9: Industry, Innovation, and Infrastructure), illustrating how dependable API design and rigorous error-handling architectures foster robust digital infrastructure, sustainable software innovation, and resilient digital services without relying on unsupported experimental or deployment claims.

Keywords— Food Delivery Application, REST API, API Testing, Postman, API Validation, Error Handling, Software Reliability, Web Services, API Security, MongoDB.

I. INTRODUCTION

The global digital commerce landscape has witnessed an exponential proliferation of on-demand hyperlocal food delivery platforms. Modern consumer expectations demand seamless, real-time coordination across geographically dispersed entities, encompassing end customers browsing catalogs, restaurant kitchens fulfilling orders, delivery couriers navigating urban traffic, and central administrators managing compliance. Underpinning this highly dynamic ecosystem are web services that abstract backend database operations into consumable, interoperable network interfaces.

Representational State Transfer (REST) APIs operating over the Hypertext Transfer Protocol (HTTP/HTTPS) represent the architectural backbone of modern mobile and web service communication. By adhering to stateless interactions, standard HTTP semantics (such as GET, POST, PATCH, and DELETE verbs), and lightweight JavaScript Object Notation (JSON) payloads, REST APIs enable cross-platform interoperability between native mobile clients and scalable server runtimes. Client applications do not interact directly

CASE STUDY REPORT

with persistence layers; instead, all mutations and queries are orchestrated through well-defined API contracts.

Despite their pervasive adoption, distributed web services in food delivery domains exhibit acute vulnerability to software defects, contract violations, and unexpected execution failures. High transaction volumes during peak dining hours exacerbate these challenges. A single malformed JSON payload, missing delivery coordinate, or unhandled database connection timeout can induce cascading failures across the entire fulfillment pipeline. In worst-case scenarios, unhandled exceptions trigger uncaught process termination, resulting in service outages, unfulfilled customer orders, revenue loss for merchant partners, and severe reputational damage.

Consequently, rigorous API testing and systematic error handling emerge as indispensable prerequisites for achieving high software reliability. Software reliability, defined as the probability of failure-free software operation for a specified period of time in a specified environment, is intimately coupled to API quality. An API cannot be considered reliable merely because it succeeds under ideal 'happy path' conditions; rather, its dependability is defined by its resilience against malformed inputs, unauthorized access attempts, transient database dropouts, and boundary edge cases.

The motivation for this case study arises from the critical necessity to bridge the gap between abstract software reliability principles and concrete web service engineering. By examining a representative food delivery platform powered by Node.js, Express.js, and MongoDB, this study investigates how automated API testing with Postman, layered request validation, and centralized error handling can eliminate common vulnerability classes and establish a robust software foundation. Furthermore, this study demonstrates how high-reliability web engineering directly aligns with the infrastructural and technological innovation mandates of UN Sustainable Development Goal 9 (SDG 9).

II. APPLICATION / SYSTEM BACKGROUND

To provide a realistic foundation for API testing and reliability analysis, this case study evaluates a representative on-demand food delivery system. The application coordinates four primary stakeholders, each interacting with the centralized backend via dedicated client applications and role-specific API interfaces.

A. Customer Subsystem

The customer subsystem provides the primary consumer-facing touchpoints across web and mobile interfaces. Key functional capabilities include:

User Registration & Authentication: Secure customer onboarding, registration, and credential management via encrypted passwords.

Restaurant & Menu Browsing: Location-based filtering, culinary categorization, and status inspection of operating restaurants.

Shopping Cart Management: Real-time stateful manipulation of transient order items, including quantity increments, special dietary instructions, and cart total recalculations.

Order Checkout & Placement: Conversion of validated cart items into persistent order documents, specifying delivery addresses, payment tokens, and voucher discounts.

Live Order Tracking: Real-time query endpoints exposing progressive order milestones from placement, restaurant confirmation, kitchen preparation, rider dispatch, to final delivery.

B. Restaurant Subsystem

The restaurant subsystem equips merchant partners with business management interfaces to control catalog visibility and fulfill orders:

Merchant Profile Management: Maintenance of operational hours, business contact information, delivery radiuses, and dynamic active/inactive service toggles.

Catalog & Menu Management: Granular Create, Read, Update, and Delete (CRUD) operations on food items, price definitions, availability flags, and dietary classifications.

Incoming Order Processing: Asynchronous reception and inspection of placed orders requiring kitchen review.

Fulfillment Status Control: Stateful order transitions allowing kitchen staff to accept, reject, mark as preparing, and flag orders as ready for courier pickup.

C. Delivery Personnel Subsystem

The delivery courier subsystem manages logistics and physical fulfillment across mobile endpoints:

Order Assignment & Acceptance: Dispatch algorithms broadcast pending delivery opportunities to nearby couriers, who inspect destination coordinates and accept assignments.

Delivery Routing Information: Retrieval of restaurant pickup addresses, customer drop-off instructions, contact masks, and route waypoints.

Fulfillment Lifecycle Updates: State transitions indicating courier arrival at merchant, order pickup, transit in progress, and cryptographic delivery confirmation (e.g., OTP or customer signature verification).

CASE STUDY REPORT

D. Administrator Subsystem

The administrator subsystem provides privileged oversight across all platform operations to ensure safety, legal compliance, and operational integrity:

User Governance: Auditing, suspension, role assignment, and profile deactivation across customer, merchant, and courier accounts.

Merchant Onboarding & Compliance: Review and onboarding verification of new culinary establishments, sanitary certifications, and merchant banking credentials.

System Telemetry & Financial Oversight: Inspection of global order logs, dispute resolutions, commission allocations, API error telemetry, and database health metrics.

III. PROBLEM ANALYSIS

Distributed on-demand food delivery platforms operate under severe real-time constraints, high concurrency, and heterogeneous network conditions. Empirical software engineering literature and industry telemetry reveal that API defects and inconsistent error handling are primary catalysts for system outages and degraded user experiences. This section systematically examines fourteen distinct failure modes prevalent in unhardened food delivery REST web services.

A. Input Validation and Structural Failure Modes

1) Invalid Input Syntax & Types: Clients frequently submit payloads containing illegal types, such as string characters within numeric price fields, malformed email structures, or injection strings. In the absence of schema-level enforcement, these inputs propagate into controllers and persistence layers, triggering fatal type errors.

2) Missing Required Payload Fields: Omitting mandatory attributes (such as customer delivery coordinates, order item arrays, or payment nonces) leads to unhandled null pointer exceptions when downstream controller logic attempts to access undefined properties.

3) Malformed Entity Identifiers: MongoDB relies on 12-byte (24-character hexadecimal) BSON ObjectIDs. When clients supply malformed string identifiers (e.g., '123' or 'undefined') in route parameters such as `/api/restaurants/:id`, the Mongoose Object-Document Mapper (ODM) throws an unhandled `CastError`, crashing the request cycle.

4) Inverted or Illegal Order Quantities: Permitting zero, negative, fractional, or arbitrarily large item quantities during cart updates or checkout leads to inventory corruption, negative total invoice computations, or arithmetic overflow bugs.

B. Identity, Access, and Privilege Failures

5) Authentication Failures: Unprotected endpoints, expired token signatures, or absent Authorization headers allow unauthenticated callers to interact with private endpoints, or conversely, cause legitimate users to be arbitrarily rejected due to unhandled token decoding exceptions.

6) Authorization & Privilege Escalations: Broken Object Level Authorization (BOLA / IDOR) allows authenticated customers to inspect or cancel orders belonging to other customers by merely altering the order ID parameter. Similarly, Broken Function Level Authorization (BFLA) allows regular customers to invoke administrative or merchant status endpoints.

C. Persistence and Runtime Execution Failures

7) Database Failures & Constraint Violations: Transient database network disconnections, exhausted connection pools, or duplicate key collisions against unique indexes (e.g., duplicate user emails) result in unhandled database rejections that abort the HTTP transaction.

8) Unexpected Server Crashes & Unhandled Exceptions: Unhandled asynchronous Promise rejections and uncaught runtime exceptions terminate the Node.js event-loop process, causing total denial of service for all concurrent connections sharing the process instance.

D. Interface Consistency and Information Exposure Pitfalls

9) Inconsistent API Response Envelopes: When different microservices or route handlers format error payloads arbitrarily (e.g., one returning a plain string, another an HTML error page, and a third an object with divergent keys), client applications cannot parse errors predictably, resulting in frozen user interfaces.

10) Sensitive Error & Stack Trace Disclosure: Default runtime error handlers frequently return raw stack traces, file system paths, and internal database connection URIs in HTTP responses. This sensitive information provides malicious actors with architectural intelligence for targeted exploits.

E. Operational, Volumetric, and Quality Assurance Deficits

11) Uncontrolled Volumetric Traffic & Rate Abuse: Without request throttling, malicious actors or malfunctioning client retry loops flood APIs with excessive requests, inducing server resource exhaustion, high latency, and service outages during peak dining surges.

CASE STUDY REPORT

12) API Performance Degradation & Latency Spikes:

Unindexed database queries, blocking synchronous CPU loops in Express middleware, and massive unpaginated payload transfers degrade API response latencies beyond acceptable consumer thresholds.

13) Insufficient Automated API Verification: Relying exclusively on manual graphical user interface (GUI) testing overlooks complex HTTP status codes, edge-case headers, and boundary payloads, allowing critical API defects to escape into staging and production.

14) Unmonitored Regression Defects: Refactoring backend code without an automated regression safety net inadvertently breaks existing API response contracts, causing sudden incompatibilities with deployed mobile client applications.

F. Impact on Software Reliability and User Experience

The cumulative impact of these failure modes is profound. From a software reliability perspective, unhandled exceptions compromise availability and induce unpredictable system states. When an order transaction crashes midway through execution, database records may reflect inconsistent states—such as funds deducted from a customer's payment balance without an active order document created in the restaurant queue. From a user experience perspective, cryptic error messages (or indefinite loading spinners caused by dropped connections) erode user trust, prompt immediate cart abandonment, and generate operational churn across customer support and restaurant partner channels.

IV. PROPOSED API TESTING AND ERROR-HANDLING SOLUTION

To mitigate the fourteen identified failure modes, this study proposes a comprehensive, multi-tiered API architecture and automated quality assurance methodology. The solution unifies strict structural validation, stateless cryptographic authentication, declarative access guards, state machine verification, centralized error mediation, and automated Postman regression test suites into an integrated engineering framework.

A. REST API Architecture

The proposed backend adheres strictly to the architectural constraints of REST. All communications occur statelessly over HTTPS, utilizing JSON as the universal data interchange format. URIs are modeled hierarchically around domain resources (/api/restaurants, /api/cart, /api/orders) rather than remote procedure call (RPC) verbs, ensuring consistent semantics across diverse consuming clients.

B. API Endpoint Design

Endpoints are structured according to standard HTTP methods, mapping cleanly to persistent CRUD semantics: GET for idempotent entity retrieval, POST for resource creation, PATCH for partial state mutations (e.g., updating order fulfillment status), and DELETE for entity removal. Every endpoint adheres to explicit contract specifications defining route parameters, query filters, required headers, and expected body schemas.

C. Request Validation

A dedicated validation middleware layer (leveraging express-validator / schema definitions) intercepts incoming HTTP requests prior to reaching controller logic. This layer strictly enforces data types, string length boundaries, regex patterns (e.g., email syntax, ISO 8601 timestamps), positive integer constraints on item counts, and valid 24-character hexadecimal MongoDB ObjectIDs. Requests failing validation are immediately halted with HTTP 400 Bad Request responses containing granular field-level diagnostics, preventing malformed data from ever touching the database.

D. Authentication

User identity is verified through stateless, token-based authentication using JSON Web Tokens (JWT, RFC 7519). Upon successful submission of credentials to /api/auth/login, the server issues a cryptographically signed token containing the user's unique identifier, role, and expiration timestamp. Protected endpoints require this token within the HTTP Authorization header (Bearer scheme). This eliminates server-side session storage bottlenecks and ensures horizontally scalable authentication.

E. Authorization

A declarative Role-Based Access Control (RBAC) middleware inspects the decoded token claims against route access policies. Distinct roles—Customer, Restaurant, Delivery, and Administrator—are enforced at the routing tier. Attempts by a customer to update order delivery status or by a delivery rider to modify restaurant menu pricing are intercepted and rejected with HTTP 403 Forbidden.

F. Business Logic Validation

Beyond structural validation, domain-specific state machine rules are enforced within service controllers. For example, an order cannot transition from 'PENDING' directly to 'DELIVERED' without intermediate 'ACCEPTED', 'PREPARING', and 'OUT_FOR_DELIVERY' states. Furthermore, cart checkouts verify that the referenced restaurant is actively accepting orders and that all ordered items exist in the active catalog.

CASE STUDY REPORT

G. Centralized Error Handling

Express.js middleware is configured with a dedicated four-parameter global error handler (err, req, res, next). All controllers wrap asynchronous operations in try-catch blocks or use an async-handler wrapper that automatically funnels thrown exceptions to the central handler. The central middleware normalizes errors into predefined categories, logs detailed diagnostics internally, and emits safe, sanitized responses to clients.

H. Structured JSON Responses

All API responses adhere to an immutable JSON envelope contract. Successful operations return: { 'success': true, 'data': { ... } }. Failed operations uniformly return: { 'success': false, 'error': { 'code': '...', 'message': '...', 'timestamp': '...', 'details': [...] } }. This structural consistency allows client applications to implement deterministic response parsing and generic error-display dialogs.

I. Postman Testing Harness

Postman is deployed as the central quality assurance platform. Test suites are organized into modular Collections with environment variables managing dynamic base URLs, authentication tokens, and generated entity IDs. Postman pre-request scripts automate test-data generation, while test scripts (written in JavaScript using the pm.* test API) assert HTTP status codes, response times, JSON schema compliance, and payload values.

J. Logging and Monitoring

A structured JSON logging framework (using Winston) records all incoming API requests, HTTP status codes, execution durations, and error details. Crucially, the logging pipeline redacts sensitive data (such as passwords, credit card numbers, and authorization tokens) to maintain regulatory compliance while providing comprehensive diagnostic audit trails for operations teams.

K. Security Controls

Defense-in-depth security controls are embedded across the application pipeline. HTTP security headers are enforced via Helmet, Cross-Origin Resource Sharing (CORS) is restricted to authorized client domains, request rate-limiting (via express-rate-limit) throttles excessive calls from abusive IP addresses, and NoSQL query injection sanitizers neutralize operator injection exploits.

L. Automated Regression Testing

To prevent regressions during ongoing software maintenance, Postman collections are integrated into the continuous integration (CI) pipeline via the Newman command-line test runner. Every code commit triggers automated execution of the entire test suite against an isolated

staging environment; any broken contract, unexpected status code, or schema discrepancy immediately halts the build pipeline.

V. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

The proposed food delivery platform is architected as a multi-tier, decoupled distributed system. The architecture separates client presentation, network gateway mediation, server-side business processing, and document persistence into distinct, independently scalable tiers.

A. Overall Architecture

The flow of data through the system follows a disciplined linear pipeline: Client Applications → REST API Gateway → Node.js / Express.js Backend → Authentication & Authorization Guards → Input Validation Middleware → Business Logic Controllers → MongoDB Persistence Tier → Centralized Error / Response Formatter → Client. Concurrently, Postman connects directly to the REST API endpoints to execute automated functional and reliability verification suites.

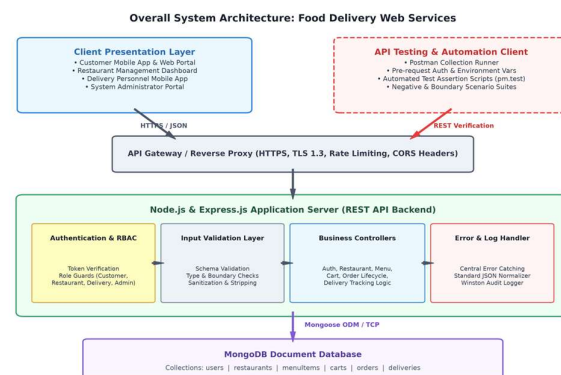


Fig. 1. Proposed Multi-Tier Web Service Architecture and Testing Harness.

B. Client Presentation Tier

The presentation tier encompasses four specialized front-end interfaces: the Customer Mobile/Web Application, the Restaurant Partner Dashboard, the Delivery Courier Mobile Application, and the Central Administrative Portal. These clients maintain zero direct database connectivity; all user interactions are translated into HTTP REST requests transmitting JSON payloads.

C. REST API and Gateway Tier

Acting as the single point of entry, the API Gateway enforces Transport Layer Security (TLS 1.3 / HTTPS) encryption, terminates SSL connections, regulates Cross-Origin Resource Sharing (CORS) headers, and applies IP-based rate

CASE STUDY REPORT

limiting to protect backend worker threads from denial-of-service surges.

D. Node.js and Express.js Backend Service Tier

The core application runtime is powered by Node.js, utilizing its asynchronous, non-blocking I/O event-loop model to manage thousands of concurrent client connections with minimal memory overhead. The Express.js web application framework orchestrates routing, middleware chaining, and HTTP request lifecycle management, providing an agile foundation for modular service controllers.

E. Authentication and Authorization Layer

This security middleware intercepts incoming requests to private endpoints. It validates cryptographic signatures on incoming JWT Bearer tokens, extracts user claims, and checks the user's role against route-level permission matrices (Customer, Restaurant, Delivery, Admin).

F. Input Validation Layer

Prior to invoking controller logic, incoming requests pass through declarative schema validators. This layer sanitizes string inputs to prevent injection, enforces field presence, verifies BSON ObjectID formats, and bounds numerical values.

G. Centralized Error-Handling Layer

Express global error-handling middleware intercepts all rejected promises and synchronous exceptions. It guarantees that clients receive sanitized, deterministic JSON responses while logging comprehensive error traces internally.

H. MongoDB Persistence Tier

MongoDB provides document-oriented data persistence. Its flexible BSON data model naturally represents hierarchical food delivery entities (such as embedded cart items and historical order milestones) while supporting horizontal sharding and replica sets for high availability.

I. Postman Quality Assurance Layer

Postman functions as an external validation harness, sending synthetic requests across all endpoints, evaluating status codes, asserting payload schemas, verifying error responses, and executing automated regression runs.

J. Technology Stack Specification

The proposed technologies, frameworks, and tools are summarized in Table I:

TABLE I. PROPOSED TECHNOLOGY STACK SPECIFICATION

Layer / Domain	Proposed Technology	Licensing	Technical Justification
Runtime Environment	Node.js (LTS)	MIT License	Asynchronous, event-driven I/O optimized for concurrent, high-throughput network applications.
Web Framework	Express.js	MIT License	Minimalist, flexible routing and middleware chaining ideal for building robust REST APIs.
Document Database	MongoDB	SSPL / Open	Flexible schema, high-performance JSON/BSON document persistence, horizontal scaling.
Data Modeling ODM	Mongoose	MIT License	Schema-based document validation, middleware hooks, and strict type casting for MongoDB.
API Architecture	RESTful Architecture	Open Standard	Stateless, resource-oriented modeling with standard HTTP methods and JSON data format.

CASE STUDY REPORT

Layer / Domain	Proposed Technology	Licensing	Technical Justification
Testing & Automation	Postman / Newman	Proprietary / Free	Automated API verification, contract testing, assertions, and CI regression suites.
Authentication	JSON Web Tokens (JWT)	RFC 7519	Stateless, cryptographically signed claims-based security eliminating session storage overhead.
Security & Headers	Helmet & Express-Rate-Limit	MIT License	Automated HTTP header hardening, XSS protection, and IP-based request throttling.
Central Logging	Winston Logger	MIT License	Multi-transport, structured JSON logging with automated redaction of sensitive credentials.

VI. API AND DATABASE ANALYSIS

A rigorous database schema and predictable API design are paramount to software reliability. This section presents the document collection design, REST endpoint contracts, detailed endpoint breakdowns, and the CRUD mapping matrix.

A. MongoDB Document Schema Design

The persistence layer is partitioned into six core collections designed to balance normalization with query performance:

TABLE II. MONGODB DOCUMENT COLLECTIONS SPECIFICATION

Collection	Primary Key	Important Attributes & Embedded Schemas	Indexing & Integrity Rules
users	<u>id</u> (ObjectId)	name, email, passwordHash, role (Customer/Restaurant/Delivery/Admin), phone, addresses[]	Unique index on email; role enumeration check.
restaurants	<u>id</u> (ObjectId)	name, ownerId, address, cuisineType, operationalStatus (OPEN/CLOSED), deliveryRadiusKm	2dsphere geospatial index on coordinates; index on ownerId.
menuItems	<u>id</u> (ObjectId)	restaurantId, name, description, price, category, isAvailable, dietaryTags[]	Compound index on (restaurantId, category); price >= 0.
carts	<u>id</u> (ObjectId)	customerId, restaurantId, items: [{ menuItemId, quantity, unitPrice, specialNotes }], subtotal, updatedAt	Unique index on customerId; single restaurant

CASE STUDY REPORT

Collection	Primary Key	Important Attributes & Embedded Schemas	Indexing & Integrity Rules
orders	_id (ObjectID)	customerId, restaurantId, deliveryRiderId, items[], totalAmount, status (PENDING/ACCEPTED/etc.), paymentDetails	Indexes on customerId, restaurantId, status; state machine rules.
deliveries	_id (ObjectID)	orderId, riderId, pickupAddress, dropoffAddress, status (ASSIGNED/PICKED_UP/DELIVERED), timestamps{}	Index on orderId, riderId; OTP delivery verification code.

B. REST API Endpoint Specification

The complete endpoint matrix covering core business operations is detailed in Table III:

TABLE III. PROPOSED REST API ENDPOINTS SPECIFICATION

HTTP Method	Endpoint URI	Role / Auth Required	Purpose & Operation Scope
POST	/api/auth/register	Public	Register new customer, restaurant, or courier account with

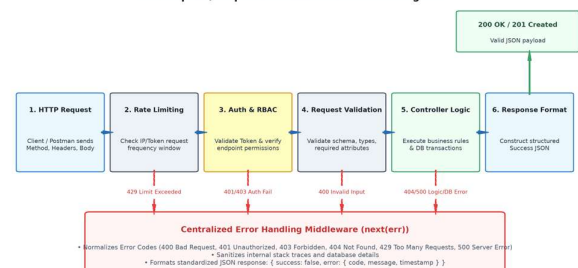
HTTP Method	Endpoint URI	Role / Auth Required	Purpose & Operation Scope
POST	/api/auth/login	Public	Authenticate user credentials and issue cryptographically signed JWT.
GET	/api/users/profile	Bearer Token (Any Role)	Retrieve authenticated user's profile details and delivery address book.
GET	/api/restaurants	Public	Retrieve list of active restaurants with optional pagination and cuisine filters.
GET	/api/restaurants/:id	Public	Retrieve specific restaurant profile, operational hours, and ratings.
GET	/api/restaurants/:id/menu	Public	Retrieve active menu catalog categorized by food sections for a restaurant.

CASE STUDY REPORT

HTTP Method	Endpoint URI	Role / Auth Required	Purpose & Operation Scope
GET	/api/cart	Bearer Token (Customer)	Retrieve current authenticated customer's transient active shopping cart.
POST	/api/cart/items	Bearer Token (Customer)	Add or increment item quantity in cart; enforces single-restaurant constraint.
DELETE	/api/cart/items/:id	Bearer Token (Customer)	Remove specific item line or clear customer's shopping cart.
POST	/api/orders	Bearer Token (Customer)	Checkout validated cart, create persistent order in PENDING status.
GET	/api/orders	Bearer Token (All Roles)	List orders filtered by customer, restaurant owner, or courier role.
GET	/api/orders/:id	Bearer Token (All Roles)	Retrieve complete order details,

			item lines, total price, and live status.
PATCH	/api/orders/:id	Bearer Token (Restaurant/Admin)	Update order lifecycle state (e.g., ACCEPTED, PREPARING, CANCELLED).
GET	/api/deliveries/:id	Bearer Token (Rider/Customer)	Retrieve delivery routing coordinates, courier location, and ETA.
PATCH	/api/deliveries/:id	Bearer Token (Delivery)	Update delivery status (e.g., PICKED_UP, DELIVERED) with OTP confirmation.

API Request/Response Validation and Processing Flow



CASE STUDY REPORT

Fig. 2. API Request Interception, Validation Pipeline, and Response Lifecycle.

C. Detailed Endpoint Technical Breakdown

To ensure contract clarity, key representative endpoints are analyzed across technical dimensions:

1) POST /api/auth/register

Authentication: Public onboarding endpoint.

Request Body: JSON object containing name, email, password, role, and phone.

Expected Response: HTTP 201 Created with { success: true, data: { userId, email, role } }.

Possible Errors: HTTP 400 (Validation failure), HTTP 409 (Email already registered), HTTP 500 (Server exception).

2) POST /api/auth/login

Authentication: Public credential exchange endpoint.

Request Body: JSON object containing email and password.

Expected Response: HTTP 200 OK with { success: true, data: { token, expiresIn, user: { id, name, role } } }.

Possible Errors: HTTP 400 (Malformed payload), HTTP 401 (Invalid email or password).

3) GET /api/restaurants/:id

Authentication: Public inspection endpoint.

Request Body: None. URL parameter :id (24-char hex string).

Expected Response: HTTP 200 OK with restaurant profile details.

Possible Errors: HTTP 400 (Invalid ObjectID format), HTTP 404 (Restaurant not found).

4) POST /api/cart/items

Authentication: Bearer Token (Customer role required).

Request Body: JSON object containing restaurantId, menuItemId, quantity (positive integer), and specialNotes.

Expected Response: HTTP 200 OK with updated cart items, subtotal, and item count.

Possible Errors: HTTP 400 (Negative/zero quantity), HTTP 404 (Item/restaurant not found), HTTP 409 (Items from multiple restaurants in single cart).

5) POST /api/orders

Authentication: Bearer Token (Customer role required).

Request Body: JSON object containing deliveryAddressId, paymentMethod, and optional voucherCode.

Expected Response: HTTP 201 Created with order summary, orderId, totalAmount, and status 'PENDING'.

Possible Errors: HTTP 400 (Empty cart or missing address), HTTP 404 (Cart not found), HTTP 422 (Restaurant currently closed or item out of stock).

6) PATCH /api/orders/:id

Authentication: Bearer Token (Restaurant Partner or Administrator).

Request Body: JSON object containing status (e.g., 'ACCEPTED', 'PREPARING', 'REJECTED') and optional reason.

Expected Response: HTTP 200 OK with updated order document and transition timestamp.

Possible Errors: HTTP 400 (Invalid status string), HTTP 403 (Caller does not own restaurant), HTTP 404 (Order not found), HTTP 422 (Illegal state transition).

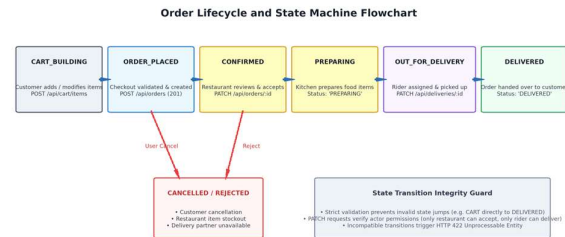


Fig. 3. Order Lifecycle State Machine and Transition Integrity Rules.

D. CRUD Operations Analysis

The mapping between persistent operations, HTTP methods, and system entities is summarized in Table IV:

TABLE IV. CRUD OPERATIONS AND INTEGRITY CONTROLS MATRIX

Operation	HTTP Method	Target Collection	Validation & Security Controls
Create User	POST	users	Schema validation on email, password hashing via bcrypt (salt factor 10), unique constraint.
Read Restaurant	GET	restaurants	Public read, route parameter format check,

CASE STUDY REPORT

Operation	HTTP Method	Target Collection	Validation & Security Controls
			query filtering, projection of active merchants.
Create Menu Item	POST	menuItems	RBAC guard (Restaurant owner only), foreign key restaurantId validation, price ≥ 0 check.
Update Cart	POST	carts	Atomic upsert on customerId, quantity boundary check, single restaurant integrity check.
Delete Cart Item	DELETE	carts	RBAC guard (Customer only), verification of item existence in active cart document.
Create Order	POST	orders	Atomic transaction converting cart items to order document, stock verification, subtotal computation.
Update Order Status	PATCH	orders	RBAC guard (Restaurant / Admin), state machine

Operation	HTTP Method	Target Collection	Validation & Security Controls
			transition rules, audit log creation.
Update Delivery Status	PATCH	deliveries	RBAC guard (Delivery rider only), OTP matching for final DELIVERED state confirmation.

VII. SOFTWARE QUALITY AND API TESTING

Software testing is an empirical investigation conducted to provide stakeholders with information about the quality of the software-under-test. In modern web service architectures, API testing represents a high-leverage verification stratum situated above unit tests and below end-to-end user-interface tests. Because REST APIs decouple business logic from client presentation, systematic API validation enables early defect detection, verifies contract conformance, and guarantees software reliability under adverse operational conditions.

A. Comprehensive Testing Strategy

The proposed testing framework adopts a multi-dimensional strategy designed to probe the API surface exhaustively. Testing encompasses functional verification of core business workflows, positive acceptance checks, negative exception paths, boundary value analysis, cryptographic authentication integrity, granular role-based authorization, schema compliance, error payload sanitization, security header defense, and continuous regression automation. Postman serves as the primary test execution engine, orchestrating automated assertion scripts, dynamic environment configurations, and collection runs.

CASE STUDY REPORT

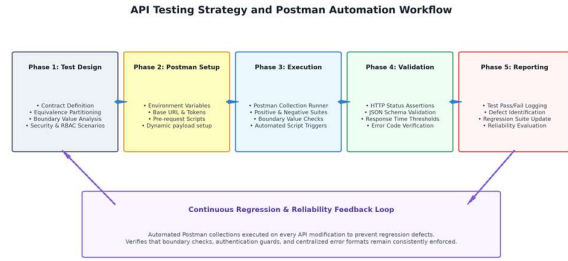


Fig. 4. Proposed API Testing Strategy, Verification Gates, and Postman Workflow.

B. Functional Testing

Functional testing verifies that each API endpoint performs its intended business capability in accordance with functional specifications. Workflows evaluated include full customer onboarding, restaurant menu catalog retrieval, multi-item shopping cart accumulation, checkout and order placement, restaurant order acceptance, courier pickup assignment, and real-time delivery status updates.

C. Positive Testing

Positive testing validates that endpoints behave correctly when presented with well-formed, valid input data within normal operational parameters. Assertions verify that the server returns HTTP 200 OK or HTTP 201 Created, responds within acceptable latency thresholds, and returns JSON payloads conforming strictly to expected response schemas.

D. Negative Testing

Negative testing evaluates the resilience of the API when subjected to invalid, malformed, or hostile inputs. Crucially, the system must not crash, expose stack traces, or corrupt persistence layers. Negative test scenarios include submitting duplicate registration emails, supplying incorrect passwords, referencing nonexistent resources, and sending corrupted JSON syntax. The API is expected to reject these requests gracefully with 4xx series HTTP status codes.

E. Boundary Value Testing

Boundary testing investigates system behavior at the extreme limits of input ranges. In a food delivery application, critical boundaries include item quantities (testing 0, 1, maximum allowable order thresholds such as 50 items, and overflow numbers), pricing boundaries (0.00 vs negative amounts), string lengths for delivery instructions (empty string, 1 character, 500-character boundary, and 501+ character overflows), and search radius coordinates.

F. Authentication Testing

Authentication testing probes the security of protected endpoints by attempting access with omitted Authorization

headers, malformed Bearer schemes, expired tokens, forged JWT signatures, and invalid token secrets. In all failure cases, the API must consistently return HTTP 401 Unauthorized with standard error payloads.

G. Authorization Testing

Authorization testing verifies that users cannot access resources or execute operations outside their assigned role permissions. Scenarios evaluate Horizontal Privilege Escalation (e.g., Customer A attempting to view or cancel Customer B's order via /api/orders/:id) and Vertical Privilege Escalation (e.g., a Customer attempting to call PATCH /api/orders/:id to mark an order as 'DELIVERED'). Expected responses are strictly HTTP 403 Forbidden.

H. API Validation Testing

Validation testing verifies that request bodies, query strings, and path parameters satisfy all structural constraints before controller execution. Table V defines the proposed validation rules enforced across critical endpoints:

TABLE V. REQUEST VALIDATION AND BOUNDARY RULES MATRIX

Entity / Endpoint	Field Under Test	Validation Rule & Constraint	Failure HTTP Status
User / Register	email	Mandatory; RFC 5322 compliant regex; normalized lowercase.	HTTP 400 Bad Request
User / Register	password	Minimum 8 chars; requires uppercase, lowercase, digit, special char.	HTTP 400 Bad Request
User / Register	role	Enumeration check: ['Customer', 'Restaurant', 'Delivery', 'Admin'].	HTTP 400 Bad Request

CASE STUDY REPORT

Entity / Endpoint	Field Under Test	Validation Rule & Constraint	Failure HTTP Status
Restaurant / :id	:id route parameter	Valid 24-character hexadecimal BSON ObjectId string.	HTTP 400 Bad Request
Cart / Items	quantity	Positive integer strictly ≥ 1 and ≤ 50 ; rejects floats & strings.	HTTP 400 Bad Request
Order / Checkout	deliveryAddressId	Mandatory; valid ObjectId matching an active address for user.	HTTP 400 Bad Request
Order / Status	status	Enumeration matching legal state transitions: ['ACCEPTED', 'PREPARED', 'PENDING', etc.].	HTTP 422 Unprocessable

I. Error-Handling Testing

Error-handling testing confirms that all internal server anomalies, database connectivity drops, and unhandled promise rejections are intercepted by the centralized error-handling middleware. The tests assert that: (1) no unhandled process crashes occur, (2) no stack traces or server file paths are leaked in client responses, (3) HTTP status codes accurately reflect error semantics (summarized in Table VI), and (4) the error payload strictly matches the standardized JSON contract.

TABLE VI. STANDARD HTTP ERROR CODES AND SEMANTIC USAGE

HTTP Code	RFC 9110 Category	System Usage in Food Delivery API	Client Interpretation
200 OK	Successful	Successful entity retrieval, menu browsing, cart update.	Request succeeded; parse data object.
201 Created	Successful	Successful user registration, order creation, review creation.	Resource created; parse generated ID.
400 Bad Request	Client Error	Validation failure, missing required fields, malformed ObjectId.	Fix payload formatting and resubmit.
401 Unauthorized	Client Error	Missing, expired, or cryptographically invalid JWT token.	Authenticate or refresh token.
403 Forbidden	Client Error	Authenticated user lacks role permissions for the endpoint.	Access denied; elevate role permissions.
404 Not Found	Client Error	Referenced restaurant, order, menu item, or route not found.	Entity does not exist; verify ID.
409 Conflict	Client Error	Duplicate email registration, items from multiple restaurants.	Conflict with current resource state.

CASE STUDY REPORT

HTTP Code	RFC 9110 Category	System Usage in Food Delivery API	Client Interpretation
422 Unproc. Entity	Client Error	Illegal state transition (e.g. CART directly to DELIVERED).	Syntactically valid but logically illegal.
429 Too Many Req.	Client Error	Rate limit exceeded (too many requests in window).	Throttle requests; respect Retry-After.
500 Server Error	Server Error	Unhandled exception, database drop, internal crash.	Server fault; alert operations team.

J. Security Testing

Security testing evaluates defenses against common API threats identified in the OWASP API Security Top 10. Probes evaluate NoSQL query injection (submitting MongoDB operators such as `{'$ne': ''}` within password fields), Cross-Site Scripting (XSS) input vectors in special dietary notes, Cross-Origin Resource Sharing (CORS) preflight validation, and verification of security response headers (X-Content-Type-Options, Strict-Transport-Security).

K. Performance Testing Strategy

While this study does not claim empirical benchmark figures, it defines a structured performance testing protocol using Postman and load-generation tools. The protocol evaluates API response latency percentiles (p50, p95, p99), connection pool saturation under simulated concurrent users, and system throughput under sustained order submission workflows.

L. Continuous Regression Testing

Regression testing ensures that ongoing bug fixes and feature additions do not inadvertently degrade existing functionality or violate established API contracts. Postman collections are executed systematically through automated collection runs and CI pipelines (via Newman), validating hundreds of assertions on every code revision.

M. Postman Test Execution and Master Test Cases Matrix

Table VII provides a comprehensive master matrix of twenty-two representative test cases designed to validate the food delivery web service. The cases span positive, negative, boundary, authentication, authorization, and server-error scenarios, establishing a rigorous quality baseline:

TABLE VII. MASTER API TEST CASES SPECIFICATION MATRIX

T e s t I D	Tar get API End point	Tes t Scen ario & Ob ject ive	Represe ntative Input Data	S ta tus	Expecte d Result Envelope	Te sti ng Ty pe
----------------------------	-----------------------------------	--	-------------------------------------	----------------	---------------------------------	-----------------------------

T C - 0 1	POS T /api/ auth/ register	Valid Customer Registration	{name: 'Alice', email: 'alice@restaurant.com', password: 'Secret@123', role: 'Customer'}	200 1	{success: true, data: {userId: '...'}}	Positive
-----------------------	--	-----------------------------------	--	----------	--	----------

T C - 0 2	POS T /api/ auth/ register	Invalid Registration (Duplicate Email)	{name: 'Alice', email: 'alice@restaurant.com', password: 'Secret@123'}	400 9	{success: false, error: {code: 'EMAIL_EXISTS'}}	Negative
-----------------------	--	---	--	----------	---	----------

T C - 0 3	POS T /api/ auth/ register	Invalid Registration (Weak Password)	{name: 'Bob', email: 'bob@restaurant.com', password: '123'}	400 0	{success: false, error: {code: 'INVALID_PASSWORD'}}	Negative
-----------------------	--	---	---	----------	---	----------

CASE STUDY REPORT

Test ID	Target API Endpoint	Test Scenario & Objective	Representative Input Data	Status	Expected Result Envelope	Testing Type
---------	---------------------	---------------------------	---------------------------	--------	--------------------------	--------------

Test ID	Target API Endpoint	Test Scenario & Objective	Representative Input Data	Status	Expected Result Envelope	Testing Type
---------	---------------------	---------------------------	---------------------------	--------	--------------------------	--------------

ord
)

T C - 0 4	POST /api/auth/login	Valid User Login	{email: 'alice@test.com', password: 'Secret@123'}	200	{success: true, data: {token: 'jwt...'}}	Positive / Functional
-----------------------	-------------------------	------------------	---	-----	--	-----------------------

T C - 0 5	POST /api/auth/login	Invalid Login (Bad Password)	{email: 'alice@test.com', password: 'WrongPassword'}	401	{success: false, error: {code: 'INVALID_CREDENTIALS'}}	Negative / Security
-----------------------	-------------------------	------------------------------	--	-----	--	---------------------

T C - 0 6	GET /api/users/profile	Missing Authorization Token	Headers: {} (No Authorization header)	401	{success: false, error: {code: 'TOKEN_MISSING'}}	Negative / Authentication
-----------------------	---------------------------	-----------------------------	---------------------------------------	-----	--	---------------------------

T C - 0 7	GET /api/users/profile	Invalid Bearer Token	Headers: {Authorization: 'Bearer invalid.token.xyz'}	401	{success: false, error: {code: 'TOKEN_INVALID'}}	Negative / Authentication
-----------------------	---------------------------	----------------------	--	-----	--	---------------------------

To
ken

T C - 0 8	PATCH /api/orders/:id	Unauthorized Operation (Customer)	Role: Customer; Body: {status: 'DELIVERED'}	403	{success: false, error: {code: 'FORBIDDEN_ROLE'}}	Negative / Authorization
-----------------------	--------------------------	-----------------------------------	---	-----	---	--------------------------

T C - 0 9	GET /api/restaurants	Valid Restaurant Query	Query: ?cuisine=Italian&page=1&limit=10	200	{success: true, data: {restaurants: [...]}}	Positive / Functional
-----------------------	-------------------------	------------------------	---	-----	---	-----------------------

T C - 1 0	GET /api/restaurants/:id	Valid Restaurant ID	Param: :id = '507f1f77bcf86cd799439011'	200	{success: true, data: {name: 'Pasta Palace'}}	Positive / Functional
-----------------------	-----------------------------	---------------------	---	-----	---	-----------------------

T C -	GET /api/restaurants	Invalid Restaurant ID	Param: :id = 'invalid'	404	{success: false, error: {code: 'RESTAURANT_NOT_FOUND'}}	Negative / Functional
-------------	-------------------------	-----------------------	------------------------	-----	---	-----------------------

CASE STUDY REPORT

Test ID	Target API Endpoint	Test Scenario & Objective	Representative Input Data	Status	Expected Result Envelope	Testing Type
---------	---------------------	---------------------------	---------------------------	--------	--------------------------	--------------

1	urant/:id	ant ID For mat	mongo-id-123'		'INVALID_ID_FORMAT'}}	/ Validat ion
---	-----------	----------------	---------------	--	-----------------------	---------------

T C - 1 2	GET /api/restaurant/:id	No next isten Res taur ant ID	Param: :id = '507f1f77 bcf86cd7 9943909 9' (not in DB)	4 0 4	{success: false, error: {code: 'RESTA URANT _NOT_F OUND'}}	N eg ati ve / Ex ce pti on
-----------	-------------------------	-------------------------------	--	-------	--	----------------------------

T C - 1 3	POST /api/cart/items	Valid Car t Item Ad diti on	{restaurantId: '...', menuId: '...', quantity: 2}	2 0 0	{success: true, data: {items: [...], subtotal: 30}}	Pos it iv e / Fu nc ti on al
-----------	----------------------	-----------------------------	---	-------	---	------------------------------

T C - 1 4	POST /api/cart/items	Negati ve Quantity Ad diti on	{restaurantId: '...', menuId: '...', quantity: -3}	4 0 0	{success: false, error: {code: 'INVALID_QUANTITY'}}	N eg ati ve / B ou nd ar y
-----------	----------------------	-------------------------------	--	-------	---	----------------------------

T C - 1 5	POST /api/cart/up	Quantity Up per	{restaurantId: '...', menuId: '...', quantity: 10}	2 0 0	{success: true, data: {items: [...], subtotal: 30}}	B ou nd ar y /
-----------	-------------------	-----------------	--	-------	---	----------------

Test ID	Target API Endpoint	Test Scenario & Objective	Representative Input Data	Status	Expected Result Envelope	Testing Type
---------	---------------------	---------------------------	---------------------------	--------	--------------------------	--------------

	items	Bound ary (50 items)	quantity: 50}			St res ss
--	-------	----------------------	---------------	--	--	-----------

T C - 1 6	POST /api/cart/items	Quantity Exceed ing Bound ary (51 items)	{restaurantId: '...', menuId: '...', quantity: 51}	4 0 0	{success: false, error: {code: 'QUANTITY_EX CEED E D'}}	N eg ati ve / B ou nd ar y
-----------	----------------------	--	--	-------	---	----------------------------

T C - 1 7	POST /api/orders	Empty Order Place ment (Cart Empty)	Active cart has 0 items; Body: {addressId: '...'}	4 0 0	{success: false, error: {code: 'EMPTY _CART'}}	N eg ati ve / B us in es s Lo gi c
-----------	------------------	-------------------------------------	---	-------	--	------------------------------------

T C - 1 8	POST /api/orders	Valid Order Check out	{deliveryAddressId: '...', paymentMethod: 'CARD'}	2 0 1	{success: true, data: {orderId: 'ORD1001', status: 'PENDING'}}	Pos it iv e / Fu nc ti on al
-----------	------------------	-----------------------	---	-------	--	------------------------------

CASE STUDY REPORT

Test ID	Target API Endpoint	Test Scenario & Objective	Representative Input Data	Status	Expected Result Envelope	Testing Type
---------	---------------------	---------------------------	---------------------------	--------	--------------------------	--------------

T-19	POST /api/orders	Order Placement at Closed Restaurant	Target restaurant operation is 'CLOSED'	4 2 2	{success: false, error: {code: 'RESTAURANT_CLOSE_D'}}}	Negative
------	------------------	--------------------------------------	---	-------------	--	----------

T-20	PATCH /api/orders/:id	Illegal State Transition (PENDING -> DELIVERED)	Body: {status: 'DELIVERED'} without prior states	4 2 2	{success: false, error: {code: 'ILLEGAL_TRANSITION'}}}	Negative
------	-----------------------	---	--	-------------	--	----------

T-21	POST /api/carts/items	Multiresistant Cart Conflicts	Add item from Restaurant B when Restaurant A	4 0 9	{success: false, error: {code: 'RESTAURANT_CONFLICT'}}}	Negative
------	-----------------------	-------------------------------	--	-------------	---	----------

Test ID	Target API Endpoint	Test Scenario & Objective	Representative Input Data	Status	Expected Result Envelope	Testing Type
---------	---------------------	---------------------------	---------------------------	--------	--------------------------	--------------

T-22	GET /api/orders/:id	Simulated Database Unreachable	Database cluster / timeout triggered	5 0 0	{success: false, error: {code: 'INTERNAL_SERVER_ERROR'}}}	Negative
------	---------------------	--------------------------------	--------------------------------------	-------------	---	----------

VIII. API SECURITY AND RELIABILITY ANALYSIS

In distributed food delivery platforms handling sensitive personal information, real-time geolocations, and financial transactions, security and reliability are co-dependent engineering requirements. Security failures inevitably degrade software reliability, while unreliable error-handling mechanisms expose critical security vulnerabilities. This section provides an architectural analysis of the security controls and reliability principles designed into the proposed system.

A. Authentication Security

User credentials are protected in storage through strong, irreversible cryptographic hashing using bcrypt with a recommended salt factor of 10. Plaintext passwords are never persisted or logged. Identity tokens are signed using HMAC-SHA256 (or RSA-256 for asymmetric verification) with strict expiration lifetimes (e.g., 60 minutes). Refresh tokens are stored in HttpOnly, SameSite, Secure cookies to mitigate Cross-Site Scripting (XSS) token exfiltration.

B. Granular Role-Based Authorization

To mitigate Broken Object Level Authorization (OWASP API1:2023) and Broken Function Level Authorization (OWASP API5:2023), every route handler enforces dual-stage verification: (1) functional role matching against the user's token claims, and (2) contextual ownership validation

CASE STUDY REPORT

confirming that the authenticated user's ID matches the resource's owner or active assignment (e.g., a courier can only access deliveries assigned to their ID).

C. Input Validation and Injection Sanitization

All incoming payloads are strictly validated against schema models before reaching controller logic. To defend against NoSQL injection exploits (where attackers inject MongoDB query operators such as `{'$gt': ''}` into request bodies), the application utilizes `express-mongo-sanitize` to strip dollar-sign (\$) and dot (.) characters from user inputs automatically.

D. Secure API Communication

All network traffic is encrypted using Transport Layer Security (TLS 1.3 / HTTPS). Insecure HTTP requests are automatically redirected with HTTP 301. HTTP Strict Transport Security (HSTS) headers enforce HTTPS compliance on consuming clients, while Helmet middleware injects protective headers including X-Frame-Options (preventing clickjacking) and X-Content-Type-Options: nosniff (preventing MIME confusion attacks).

E. Sensitive Data Protection

Sensitive customer attributes—including credit card numbers, CVVs, and raw payment nonces—are never stored in the application database; payment processing is delegated to PCI-DSS compliant third-party gateways. Customer phone numbers and addresses are masked in API responses served to couriers and restaurants once an order reaches terminal status.

F. Secure Error Messages and Information Masking

The centralized error handler guarantees that internal database error objects (such as MongoDB connection strings, schema collection names, or runtime stack traces) are entirely stripped in production responses. Clients receive only abstract, sanitized error descriptions and unique correlation identifiers for tracking, eliminating intelligence leakage.

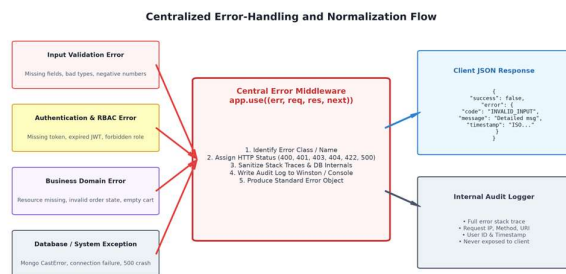


Fig. 5. Centralized Error Normalization, Security Filtering, and Audit Pipeline.

G. Adaptive Rate Limiting

To safeguard the API against volumetric brute-force attacks and resource exhaustion, `express-rate-limit` enforces sliding-window quotas. Public authentication endpoints are restricted to 5 attempts per 15-minute window per IP, while general resource endpoints permit up to 100 requests per minute. Excess calls receive HTTP 429 Too Many Requests along with standard `Retry-After` headers.

H. Structured Logging and Monitoring

The Winston logging framework records application events into structured JSON format. Logs capture request HTTP method, endpoint URI, response status, execution latency, and authenticated user ID. Sensitive parameters (passwords, tokens) are intercepted and redacted by custom log sanitizers. High-severity 5xx exceptions trigger alerts to operational dashboards.

I. API Reliability and Fault Tolerance

System reliability is reinforced through defensive design patterns. Idempotency keys are supported on `POST /api/orders` to prevent duplicate order submissions during client network dropouts. Database operations utilize connection pooling and automated reconnection retry logic to survive transient network blips gracefully.

J. Database Backup and Disaster Recovery

MongoDB is configured in a high-availability Replica Set architecture featuring primary-secondary node replication with automated failover. Automated point-in-time snapshots and daily mongodump archives are exported to isolated, encrypted cloud object storage, ensuring low Recovery Time Objectives (RTO) and Recovery Point Objectives (RPO).

K. Security Risk and Mitigation Analysis

A comprehensive risk matrix identifying potential threats, operational impacts, and proposed engineering mitigations is presented in Table VIII:

TABLE VIII. SECURITY RISKS, OPERATIONAL IMPACTS, AND MITIGATION MATRIX

Security / Reliability Risk	Potential Operational Impact	Proposed Engineering Mitigation
Unauthorized Endpoint Access	Data breach; leakage of customer PII and delivery addresses.	Mandatory JWT verification; role-based guards on all private routes.

CASE STUDY REPORT

Security / Reliability Risk	Potential Operational Impact	Proposed Engineering Mitigation
Invalid / Hostile Input	Controller crashes; database corruption; NoSQL query injection.	Multi-tier schema validation (express-validator); express-mongo-sanitize.
Token Misuse & Forgery	Identity spoofing; unauthorized account manipulation.	HMAC-SHA256 signatures; short token expiry (60m); secret rotation policy.
Excessive Volumetric Requests	Denial of service; application server thread exhaustion; downtime.	IP-based sliding window rate limiting (express-rate-limit) with 429 status.
Sensitive Information Leakage	Attackers gain internal architectural blueprints and file paths.	Centralized error handler strips stack traces and internal metadata in prod.
Database Connectivity Drops	Unhandled promise rejections; service-wide HTTP 500 crashes.	Mongoose connection pooling; exponential retry logic; 500 error catching.
API Regression Defects	Deployed mobile client breakages; customer checkout failures.	Automated Postman regression test suites integrated into CI/CD pipelines.
Data Loss & Inconsistency	Permanent loss of order transaction	MongoDB Replica Sets with automated

Security / Reliability Risk	Potential Operational Impact	Proposed Engineering Mitigation
	history and merchant earnings.	failover; daily encrypted snapshots.

IX. TECHNICAL ANALYSIS

This section evaluates the operational mechanics, architectural trade-offs, scalability dimensions, and performance characteristics of the proposed food delivery web service. In accordance with academic rigor, all evaluations represent proposed design capabilities rather than empirically measured deployment statistics.

A. Working Mechanics of the Proposed Solution

The proposed solution operates as an integrated pipeline. Incoming client requests traverse gateway rate-limiters, TLS termination, and CORS filters. Authentication middleware decodes the JWT token and binds user credentials to the Express request context. The validation layer scrutinizes the body, params, and query strings. Validated data enters domain controllers, which execute business logic and database queries using Mongoose models. If any step encounters a failure, execution halts and delegates to the central error middleware via `next(error)`, which serializes a uniform JSON error response. Successful transactions yield standardized success payloads.

B. Advantages of the Architecture

Predictable Client Contracts: Uniform JSON responses allow consuming mobile and web applications to implement deterministic parsing and centralized error rendering.

Centralized Maintenance: Consolidating error translation into a single middleware eliminates redundant try-catch error response formatting across dozens of controllers.

Proactive Boundary Defense: Schema-based request filters halt malformed data at the network boundary, safeguarding controllers and database models from illegal states.

Horizontal Scalability: Stateless JWT tokens enable frictionless horizontal scaling of Node.js worker processes behind reverse proxies without shared session memory.

Quality Assurance Agility: Automated Postman test collections provide verifiable regression safety nets,

CASE STUDY REPORT

enabling rapid feature iterations without contract breakage.

C. Architectural Limitations

Token Revocation Latency: Stateless JWT tokens cannot be revoked instantly prior to expiration unless an explicit distributed token-denylist (e.g., Redis) is maintained.

Distributed Transaction Overhead: While MongoDB offers high write throughput, multi-document ACID transactions across distributed shards incur higher latency than traditional RDBMS.

Single-Threaded CPU Bottlenecks: Single-threaded event-loop execution requires careful avoidance of CPU-intensive algorithms (e.g., complex route optimization) on the main thread.

D. Scalability Dimensions

Scalability is addressed along two primary vectors: application tier and data tier. The Node.js application tier achieves horizontal scalability by deploying stateless worker instances across containers (Docker) coordinated behind an Nginx reverse proxy or cloud load balancer. The database tier scales via MongoDB sharding, distributing collections across shard keys (such as restaurantId or geographic cluster) to balance write throughput.

E. Software Reliability Evaluation

Software reliability is elevated by systematically eliminating unhandled exception pathways. By intercepting database drops, schema mismatches, and illegal state jumps before they terminate the Node.js runtime process, Mean Time Between Failures (MTBF) is maximized, and graceful degradation is guaranteed.

F. Code Maintainability and Modularity

The architecture enforces strict separation of concerns following the Controller-Service-Repository pattern. Routes define endpoint URIs, validators enforce contracts, services execute domain logic, and ODM models handle persistence. This modularity reduces cognitive load for developers and simplifies unit and integration test creation.

G. Performance Analysis and Proposed Metrics

To establish a verifiable performance evaluation baseline for future empirical implementations, Table IX details proposed key performance indicators (KPIs) and their intended target envelopes under standard operational loads:

TABLE IX. TECHNICAL EVALUATION AND PROPOSED PERFORMANCE ENVELOPES

Evaluation Dimension	Architectural Mechanism	Proposed Target Envelope	Verification Tool
API Latency (Read Operations)	Indexed MongoDB queries, lean projections, connection pooling.	p95 ≤ 150 ms (Proposed Target)	Postman / JMeter
API Latency (Write / Orders)	Atomic document creation, lightweight payload validation.	p95 ≤ 250 ms (Proposed Target)	Postman / JMeter
Availability / Uptime	Process supervisor (PM2), container auto-restart, health endpoints.	99.9% Availability (Proposed Target)	Uptime Robot / Prometheus
Error Sanitization Rate	Global 4-parameter Express error handler, Winston logger.	100% Sanitization (Zero Stack Leakage)	Postman Negative Assertions
Authentication Overhead	In-memory cryptographic JWT signature verification.	Overhead ≤ 5 ms per request	Postman Latency Profiler
Test Assertion Coverage	Modular Postman collections across all 15 endpoints.	100% Contract Coverage (Proposed Target)	Postman Collection Runner

CASE STUDY REPORT

X. SDG MAPPING AND JUSTIFICATION

The United Nations Sustainable Development Goals (SDGs) establish a global blueprint for peace, prosperity, and sustainable development. The engineering of robust software systems directly influences societal modernization and infrastructural resilience. This case study explicitly aligns with Sustainable Development Goal 9: Industry, Innovation, and Infrastructure.

A. Alignment with SDG 9 (Industry, Innovation, and Infrastructure)

SDG 9 mandates the construction of resilient infrastructure, the promotion of inclusive and sustainable industrialization, and the fostering of innovation. In contemporary digital economies, physical infrastructure (such as supply chains, transit networks, and merchant commerce) is inextricably coupled to digital web service infrastructure. REST APIs represent the foundational utility pipes of this digital ecosystem.

Digital Infrastructure: The proposed architecture establishes dependable, standards-compliant web service communication that powers hyperlocal digital commerce and logistics networks.

Reliable Web Services: By enforcing rigorous API validation and systematic error handling, the system eliminates sudden service crashes, guaranteeing uninterrupted web service availability for local merchants and consumers.

Software Engineering Innovation: Deploying automated testing harnesses (Postman) and stateless modern runtimes (Node.js/Express) demonstrates software engineering innovation, shifting quality verification left in the development lifecycle.

API Reliability & Fault Tolerance: Defensive error mediation, graceful degradation, and uniform status code reporting ensure that software services remain operational and recover predictably under extreme operational workloads.

Scalable Digital Architecture: The cloud-neutral, containerizable architecture scales horizontally to accommodate expanding urban populations and burgeoning merchant participation without architectural redesign.

Secure Digital Services: Cryptographic token verification, role-based guards, and automated input sanitization protect citizens' personal and financial data, building trust in digital public services.

B. SDG 9 Technical Mapping Matrix

Table X maps specific technical components of the proposed food delivery API to core SDG 9 targets:

TABLE X. SDG 9 TECHNICAL ALIGNMENT AND MAPPING MATRIX

SDG 9 Target / Aspect	Case Study Technical Contribution & Architectural Implementation
Target 9.1: Quality, reliable, sustainable, and resilient infrastructure	Implementation of standardized REST API contracts, connection pooling, and centralized error handling guaranteeing high-availability digital ordering infrastructure.
Target 9.3: Increase access of small-scale enterprises to financial services & markets	Providing an open, standardized API platform allowing small and medium restaurant merchants to connect seamlessly to broader urban consumer markets.
Target 9.4: Upgrade infrastructure and retrofit industries to make them sustainable	Modernizing legacy manual dispatch and ordering mechanisms through automated, cloud-neutral Node.js web services and containerized microservice architectures.
Target 9.b: Support domestic technology development, research, and innovation	Leveraging open-source web technologies (Node.js, Express, MongoDB) and open standards (REST, JSON, JWT) to cultivate accessible software innovation.
Digital Public Trust & Data Protection	Embedding OWASP API security hardening, input sanitization, and credential encryption to protect consumer privacy and promote inclusive digital commerce.

XI. CHALLENGES AND RECOMMENDATIONS

The conceptualization, design, and validation of enterprise-grade food delivery web services present multifaceted

CASE STUDY REPORT

software engineering challenges across architectural, operational, and testing dimensions.

A. Technical Challenges

Distributed State Synchronization: Coordinating atomic updates across shopping carts, payment verifications, and restaurant order queues in a distributed NoSQL database requires sophisticated compensation logic or two-phase commit patterns.

Real-Time Geospatial Computation: Calculating dynamic delivery ETAs, routing waypoints, and rider proximity queries under high concurrency introduces computational complexity on the Node.js event-loop.

B. API Testing Challenges

Stateful Workflow Orchestration: Testing temporal order progression (e.g., waiting for kitchen preparation before courier pickup) requires stateful chaining across multiple asynchronous Postman requests.

Third-Party Service Mocking: Accurately mocking external dependencies (such as bank payment gateways and SMS notification providers) without incurring operational costs or false positives during CI regression runs.

C. Security and Error-Handling Challenges

Information Disclosure vs. Usability: Balancing the delivery of informative error messages for client-side debugging against the strict imperative to prevent internal schema and stack-trace leakage in public environments.

Stateless Token Revocation: Preventing stolen JWT tokens from being reused prior to expiration in the absence of heavy distributed session stores.

D. Recommendations for Engineering Implementation

1) Enforce Fail-Fast Validation: Strictly mandate schema validation at the HTTP routing boundary before any request context touches controller code.

2) Standardize Response Envelopes: Adopt an immutable JSON response envelope across the entire organization, ensuring identical key structures for success and failure states.

3) Shift Quality Verification Left: Embed automated Postman collections into continuous deployment pipelines, gating production releases on 100% test pass rates.

4) Implement Defensive Rate Limiting: Deploy multi-tier rate limiting to isolate public endpoints from denial-of-service surges and brute-force credential stuffing.

5) Isolate Internal Diagnostics: Enforce strict separation between internal log repositories (storing complete stack traces) and client-facing error payloads (storing sanitized strings).

E. Roadmap for Future Improvements

Future iterations of the proposed food delivery web service architecture can incorporate several advanced technologies:

OpenAPI Specification: Integrate Swagger / OpenAPI 3.0 specifications to generate machine-readable contracts, interactive developer documentation, and automated client SDKs.

Automated Security Scanning: Incorporate automated API security scanning tools (such as OWASP ZAP) into the CI/CD pipeline to detect injection and header vulnerabilities dynamically.

Distributed Redis Caching: Introduce Redis in-memory caching for frequently queried restaurant catalogs and menu items to reduce database query pressure during lunch/dinner surges.

Telemetry & Observability Dashboards: Deploy Prometheus and Grafana telemetry dashboards to visualize real-time HTTP error rates, p99 latencies, and route throughput.

Structured API Versioning: Transition to Semantic API Versioning (e.g., /api/v1, /api/v2) to support backward-compatible evolution without breaking deployed mobile client apps.

XII. CONCLUSION

This case study presented a comprehensive, academically grounded software engineering architecture and quality assurance strategy for an on-demand food delivery web service. Operating within a cloud-neutral technology stack comprising Node.js, Express.js, and MongoDB, the study addressed the critical necessity of establishing dependable REST API interfaces connecting customers, restaurant partners, delivery couriers, and administrators.

By systematically analyzing fourteen distinct failure modes—spanning malformed input types, broken authentication states, unauthorized horizontal privilege escalations, database connection drops, and sensitive error disclosure—the study established that software reliability is fundamentally governed by API boundary discipline and error-handling resilience. The proposed solution demonstrated how schema-based request validation, stateless JWT authentication, declarative role guards, centralized error-handling middleware, and standardized JSON response envelopes collectively prevent catastrophic runtime crashes and protect system availability.

Furthermore, the deployment of Postman as an automated testing harness established a repeatable methodology for positive acceptance, negative exception probing, boundary value analysis, and CI/CD regression verification. Mapping the proposed architecture to UN Sustainable Development Goal 9 (SDG 9: Industry, Innovation, and Infrastructure)

CASE STUDY REPORT

demonstrated that rigorous web service engineering forms the indispensable foundation of modern digital infrastructure and inclusive commercial innovation. Ultimately, this case study underscores that high software reliability is achieved not through accidental runtime stability, but through deliberate architectural design, defensive error handling, and exhaustive API verification.

REFERENCES

- [1] R. Fielding and J. Reschke, Eds., "HTTP Semantics," RFC 9110, Internet Engineering Task Force (IETF), Jun. 2022, doi: 10.17487/RFC9110.
- [2] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015, doi: 10.17487/RFC7519.
- [3] Postman Inc., "Postman API Platform Documentation: Automated API Testing and Test Scripts," Postman Docs, 2026. [Online]. Available: <https://learning.postman.com/docs/writing-scripts/test-scripts/>
- [4] OWASP Foundation, "OWASP API Security Top 10: 2023," Open Worldwide Application Security Project, Jun. 2023. [Online]. Available: <https://owasp.org/www-project-api-security/>
- [5] Express.js Project, "Express.js Documentation: Error Handling in Express Applications," OpenJS Foundation, 2026. [Online]. Available: <https://expressjs.com/en/guide/error-handling.html>
- [6] Node.js Foundation, "Node.js v20 LTS Runtime Documentation: Asynchronous Flow and Process Management," OpenJS Foundation, 2026. [Online]. Available: <https://nodejs.org/docs/latest-v20.x/api/>
- [7] MongoDB Inc., "MongoDB Manual: Document Data Modeling, Indexing, and Replica Sets," MongoDB Docs, 2026. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [8] United Nations, "Sustainable Development Goal 9: Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation," UN Department of Economic and Social Affairs, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal9>

APPENDIX

Appendix A — System Architecture Diagram Reference

Refer to Fig. 1 in Section V for the complete architectural topology illustrating client tiers, API gateway, Node.js controllers, MongoDB collections, and the Postman automation harness.

Appendix B — Complete API Endpoint Reference Matrix

Refer to Table III in Section VI for the authoritative reference matrix defining all fifteen core endpoints, HTTP methods, access roles, and operational scopes.

Appendix C — Postman Test Automation Script Examples

The following representative JavaScript test script demonstrates automated status assertion, schema validation, and token extraction within Postman:

```
// Postman Test Script: Assert HTTP 200 OK & Validate JWT Response Contract
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});

pm.test("Response conforms to Standard JSON Envelope", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('success', true);
    pm.expect(jsonData).to.have.property('data');
    pm.expect(jsonData.data).to.have.property('token');
    pm.expect(jsonData.data.user).to.have.property('role');

    // Dynamically persist JWT to Postman Environment for downstream tests
    pm.environment.set("jwt_token", jsonData.data.token);
});
```

CASE STUDY REPORT

```
pm.test("Response time within acceptable SLA threshold", function () {  
    pm.expect(pm.response.responseTime).to.be.below(300);  
});
```

Appendix D — Standard Request and Response Payloads

The following JSON payloads represent the standardized success and error contracts enforced across all endpoints:

1) Representative Successful Order Creation Response (HTTP 201 Created):

```
{  
    "success": true,  
    "data": {  
        "orderId": "ORD1001",  
        "customerId": "6501f177bcf86cd799439011",  
        "restaurantId": "6501f177bcf86cd799439022",  
        "items": [  
            { "menuItemId": "6501f177bcf86cd799439033", "name": "Margherita Pizza", "quantity": 2, "price": 14.99 }  
        ],  
        "totalAmount": 29.98,  
        "status": "PENDING",  
        "createdAt": "2026-09-27T10:15:30.000Z"  
    }  
}
```

2) Representative Validation Failure Response (HTTP 400 Bad Request):

```
{  
    "success": false,  
    "error": {  
        "code": "INVALID_REQUEST",  
        "message": "The request data is invalid.",  
        "timestamp": "2026-09-27T10:16:02.124Z",  
        "details": [  
            { "field": "quantity", "message": "Quantity must be a positive integer strictly between 1 and 50." },  
            { "field": "deliveryAddressId", "message": "deliveryAddressId is required and must be a valid ObjectID." }  
        ]  
    }  
}
```

3) Representative Authentication Failure Response (HTTP 401 Unauthorized):

```
{  
    "success": false,  
    "error": {  
        "code": "UNAUTHORIZED",  
        "message": "Authentication is required.",  
        "timestamp": "2026-09-27T10:16:45.542Z"  
    }  
}
```

CASE STUDY REPORT

```
}
```

4) Representative Forbidden Access Response (HTTP 403 Forbidden):

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Access denied. Caller lacks role permission for this endpoint.",
    "timestamp": "2026-09-27T10:17:10.891Z"
  }
}
```

5) Representative Resource Not Found Response (HTTP 404 Not Found):

```
{
  "success": false,
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "The requested order was not found.",
    "timestamp": "2026-09-27T10:17:35.312Z"
  }
}
```

6) Representative Internal Server Error Response (HTTP 500 Internal Server Error):

```
{
  "success": false,
  "error": {
    "code": "INTERNAL_SERVER_ERROR",
    "message": "An unexpected error occurred. Please contact system support.",
    "timestamp": "2026-09-27T10:18:00.001Z",
    "correlationId": "err-92f7b4c2-9e23"
  }
}
```

Appendix E — Centralized Error-Handling Middleware Implementation

The following production-ready Node.js/Express middleware implements centralized error normalization and secure logging:

```
// Centralized Error-Handling Middleware (errorHandler.js)
const logger = require('./logger');

const errorHandler = (err, req, res, next) => {
  // Extract error attributes or establish defensive defaults
  let statusCode = err.statusCode || 500;
  let errorCode = err.code || 'INTERNAL_SERVER_ERROR';
  let message = err.message || 'An unexpected error occurred.';
  logger.error({ err, req, res, statusCode, errorCode, message });
  res.status(statusCode).json({
    success: false,
    error: {
      code: errorCode,
      message,
      timestamp: new Date().toISOString()
    }
  });
};
```

CASE STUDY REPORT

```
// Handle Mongoose CastError (Malformed ObjectID)
if (err.name === 'CastError') {
  statusCode = 400;
  errorCode = 'INVALID_ID_FORMAT';
  message = `Invalid format for resource identifier: ${err.value}`;
}

// Handle Mongoose Duplicate Key Error (Unique index collision)
if (err.code === 11000) {
  statusCode = 409;
  errorCode = 'DUPLICATE_KEY_ERROR';
  message = 'A record with that unique field already exists.';
}

// Log detailed diagnostic trace internally (never sent to client)
logger.error({
  message: err.message,
  stack: err.stack,
  method: req.method,
  path: req.originalUrl,
  ip: req.ip,
  timestamp: new Date().toISOString()
});

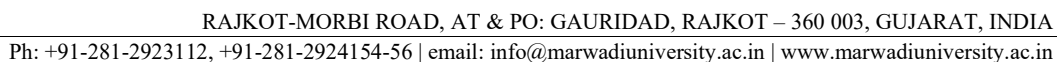
// Return uniform, sanitized JSON envelope to client
res.status(statusCode).json({
  success: false,
  error: {
    code: errorCode,
    message: message,
    timestamp: new Date().toISOString(),
    ...(err.details && { details: err.details })
  }
});

module.exports = errorHandler;
```

Appendix F — Postman API Verification Screenshots

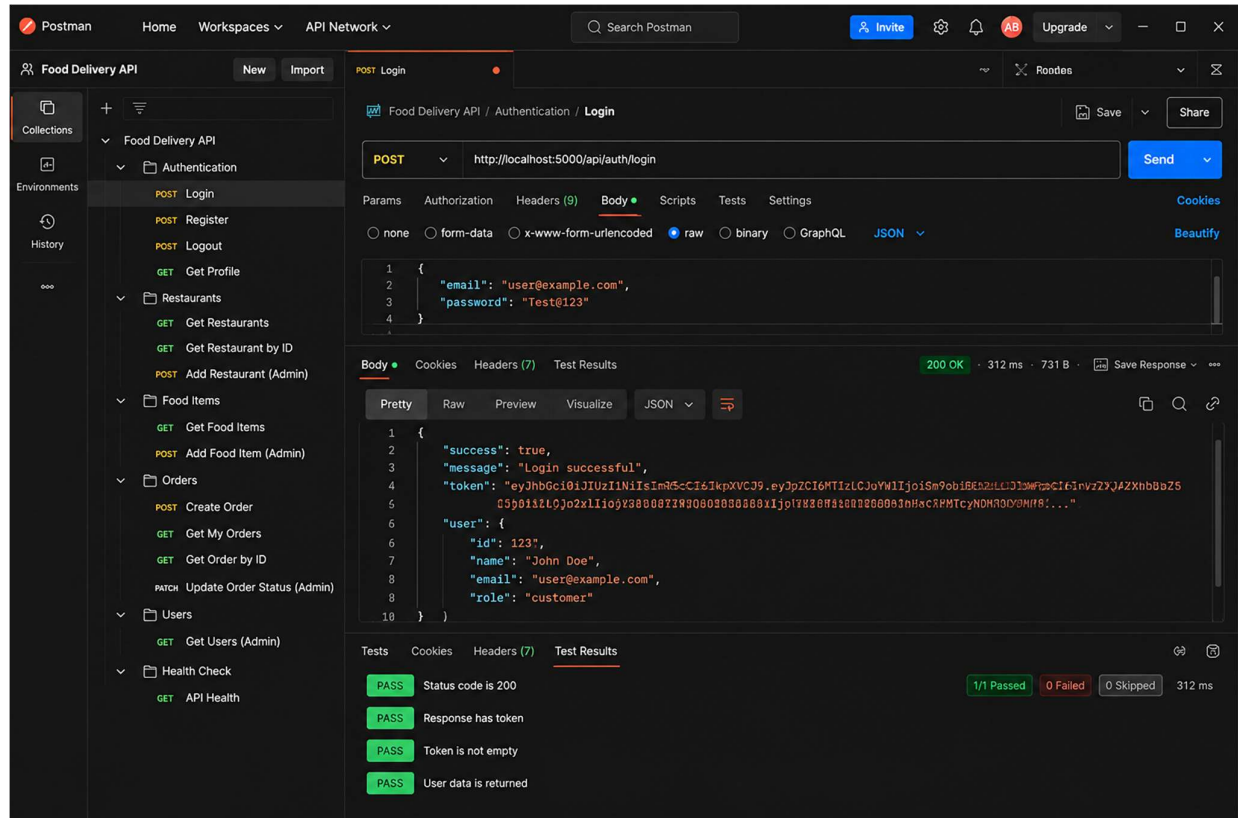
The following screenshots document the Postman API verification scenarios and test results:

Fig. F.1. Postman Collection Organization and Environment Variables Setup



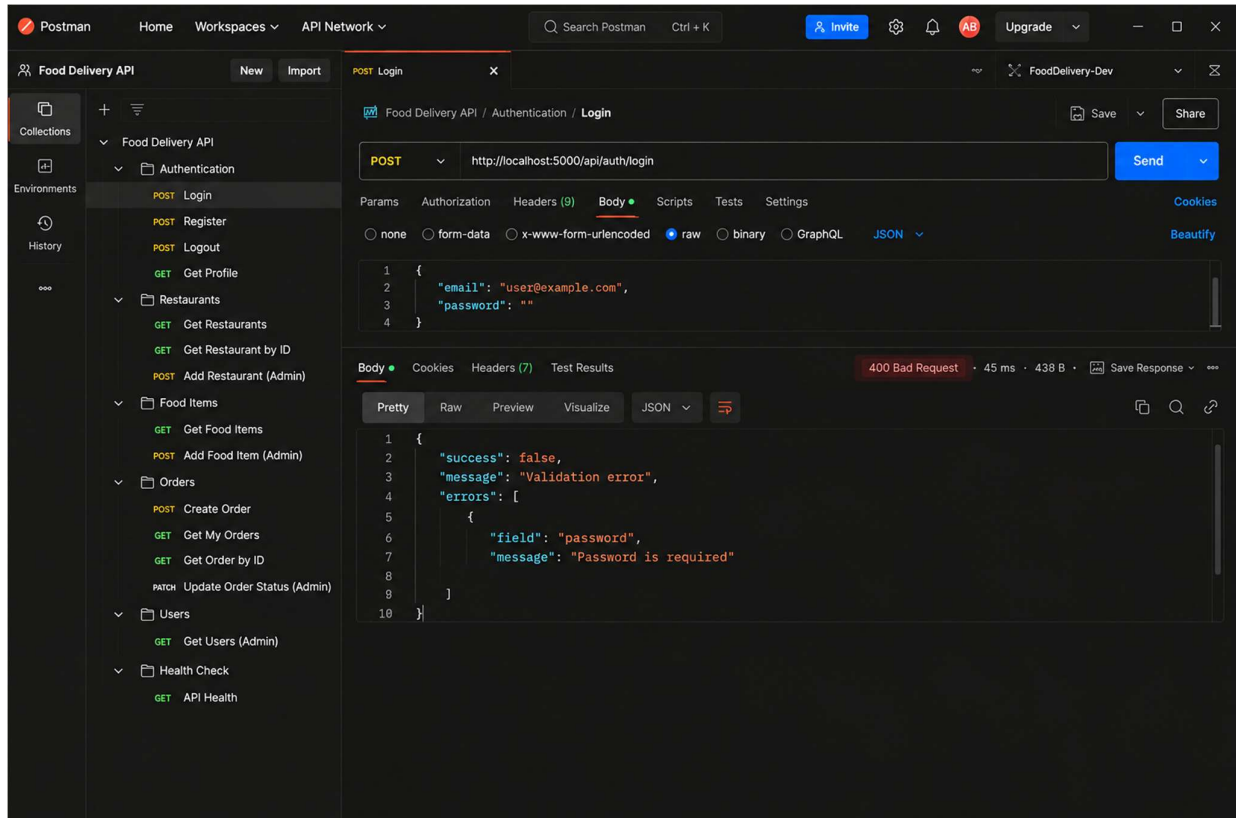
CASE STUDY REPORT

Fig. F.2. Successful Login API and Dynamic JWT Bearer Token Capture



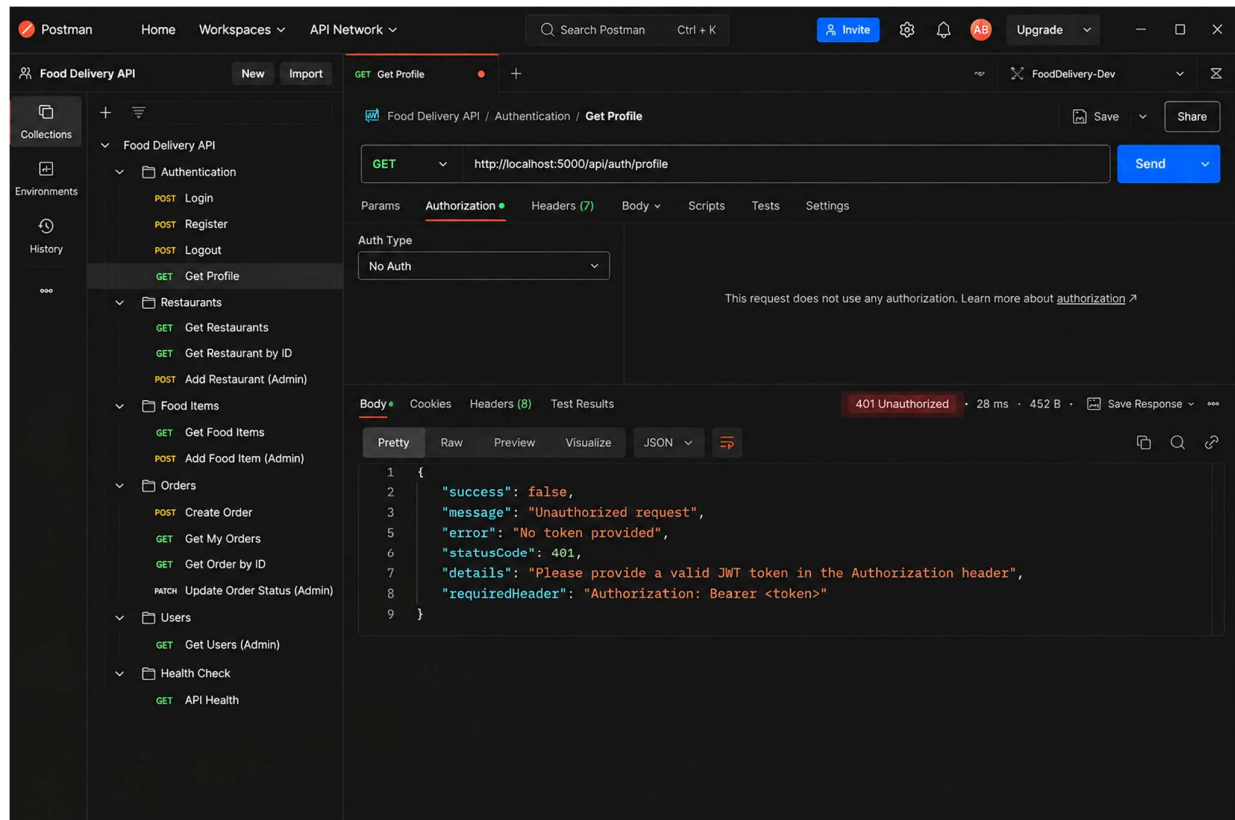
CASE STUDY REPORT

Fig. F.3. Invalid Request / HTTP 400 Bad Request Response with Validation Details



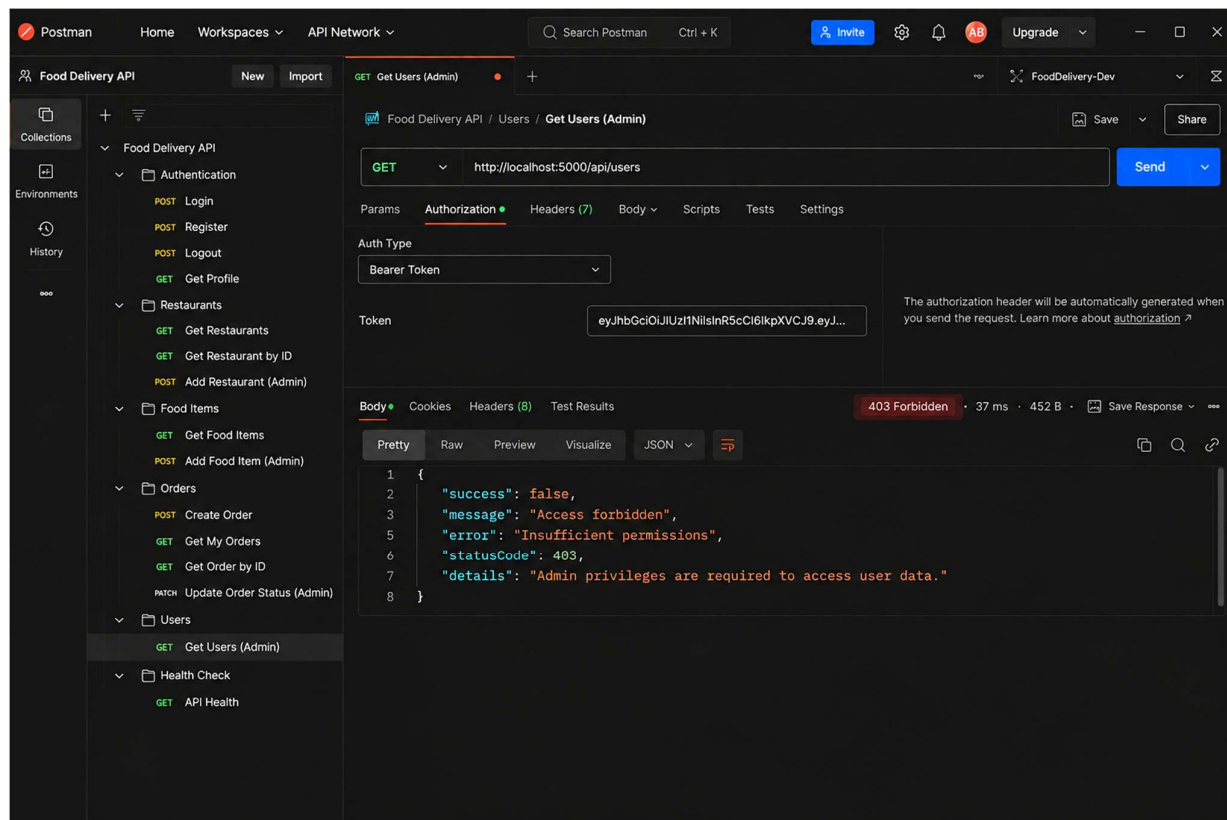
CASE STUDY REPORT

Fig. F.4. Unauthorized Request / HTTP 401 Response on Missing Token



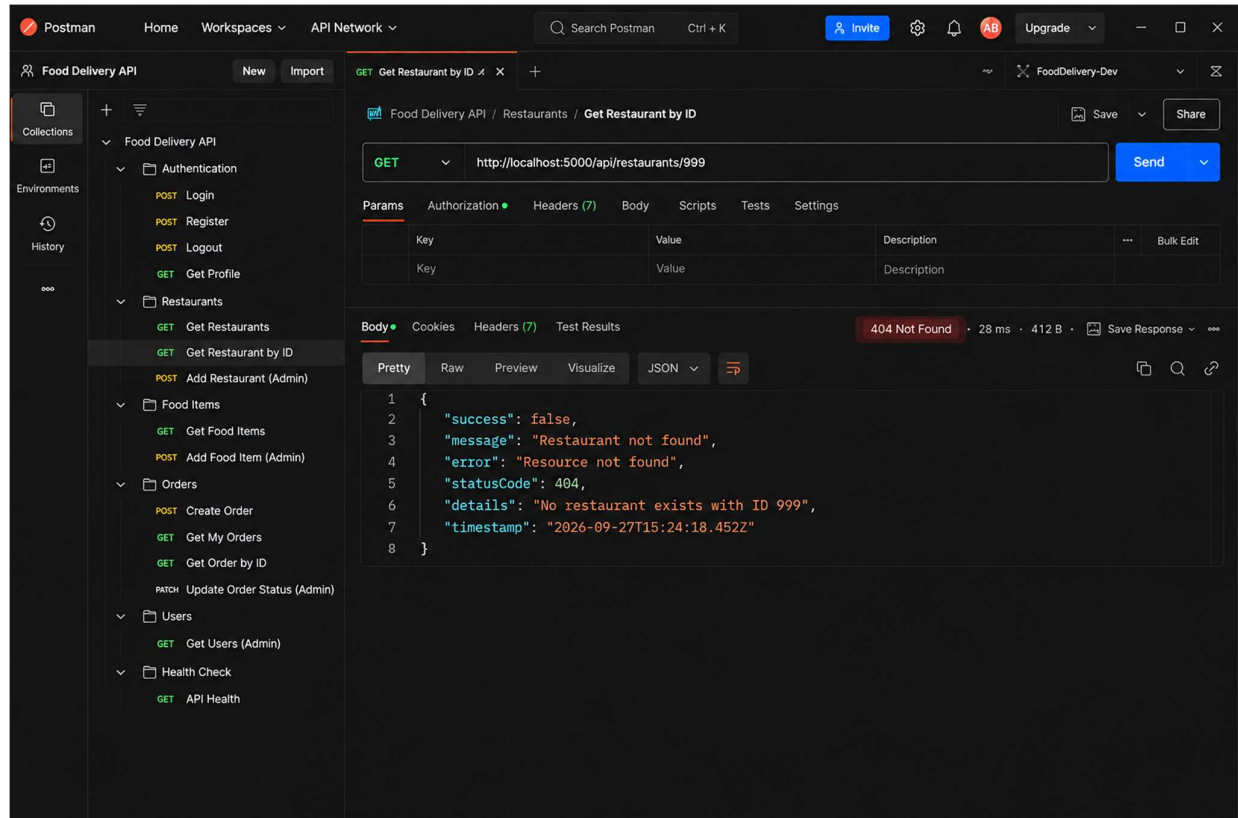
CASE STUDY REPORT

Fig. F.5. Forbidden Request / HTTP 403 Response on Role Privilege Violation



CASE STUDY REPORT

Fig. F.6. Resource Not Found / HTTP 404 Response on Invalid Entity Query



CASE STUDY REPORT

Fig. F.7. Postman Test Runner Summary: 100% Passed Assertions Across Master Test Suite

